

Full Stack Developer Assignment

Provider Mini Lead Distribution System

Purpose of This Assignment

This assignment is designed to simulate a simplified version of a real-world lead generation and distribution system similar to platforms like Prowider.

In this system:

1. A customer visits the website and submits a service enquiry
2. The system stores the enquiry as a lead
3. The lead is automatically assigned to multiple providers based on predefined business rules
4. Some providers must always receive certain service leads
5. Remaining providers should receive leads fairly over time
6. Providers should instantly see newly assigned leads in their dashboard without refreshing the page

The purpose of this assignment is to evaluate how you design and implement:

- Backend business logic
- Database consistency
- Real-time updates
- Fair lead allocation
- Concurrency handling
- Webhook/idempotency concepts
- Overall system reliability

This assignment is intentionally focused more on engineering correctness than visual UI design.

We are NOT evaluating:

- Pixel-perfect UI
- Complex styling
- Fancy animations

We ARE evaluating:

- Correctness of allocation logic
- Reliability under simultaneous requests
- Database design decisions
- Real-world backend thinking

Please keep the implementation simple, clean and reliable.

Time Expectation

Estimated effort: **6–8 hours**

Submission window: **24-48 hours**

Please keep the implementation simple and focused.

Technical Requirements

- Frontend: **Next.js**
- Database: **PostgreSQL OR MongoDB (mandatory)**
- Must provide a **live demo URL**

Not allowed:

- In-memory storage
- JSON file database
- SQLite
- Hardcoded leads
- Skipping database persistence

You may use any ORM (Prisma, Mongoose, Drizzle, etc.).

Business Scenario

A customer visits the platform and submits a service enquiry.

The system must:

1. Save the lead
2. Automatically assign the lead to specific providers
3. Follow mandatory assignment rules
4. Distribute remaining slots fairly
5. Respect provider monthly quota
6. Update dashboards in real time

This must work correctly even if multiple leads are created simultaneously.

Seed Data (Must Be Pre-Inserted)

Services

1. Service 1
 2. Service 2
 3. Service 3
-

Providers (8 Total)

Each provider has:

- Monthly quota: 10 leads

Mandatory Assignment Rules

For every new lead:

- Service 1 → Provider 1 must always receive
- Service 2 → Provider 5 must always receive
- Service 3 → Provider 1 AND Provider 4 must always receive

Each lead must be assigned to exactly **3 providers total**.

Fair Allocation Logic

After assigning mandatory providers, remaining provider slots must be distributed fairly (not random).

Pools:

- Service 1 → Providers 2, 3, 4
- Service 2 → Providers 6, 7, 8
- Service 3 → Providers 2, 3, 5, 6, 7, 8

Fair distribution must:

- Rotate over time (round-robin or equivalent)
- Persist even after server restart
- Not favor the same provider repeatedly
- Respect monthly quota

Random selection is not acceptable.

Feature 1 — Public Customer Form

Route: `/request-service`

Fields:

- Name
- Phone Number
- City
- Service Type (dropdown)
- Description

Duplicate Rule

Same phone number cannot create another lead for the SAME service.

Example:

Allowed:

- 9999999999 → Service 1
- 9999999999 → Service 2

Not allowed:

- 9999999999 → Service 1 (again)

This rule must be enforced at database level (not only frontend validation).

After submit:

- Lead must be saved
 - Provider assignment must trigger automatically
-

Feature 2 — Lead Distribution (Core Logic)

For every new lead:

1. Exactly 3 providers must be assigned
2. Mandatory providers must be included (if quota available)
3. Providers cannot exceed monthly quota (10)
4. Same provider cannot receive the same lead twice
5. Must behave correctly under simultaneous lead creation
6. Allocation state must persist in database

We will test concurrency.

Feature 3 — Provider Dashboard

Route: </dashboard>

Display for each provider:

- Remaining quota
- Leads received count
- Assigned leads list

Dashboard must show real database data.

Feature 4 — Real-Time Dashboard Update

The dashboard must automatically reflect newly assigned leads **without refreshing the page**.

Example test:

1. Keep dashboard open
2. Submit new lead in another tab
3. Dashboard should update automatically within a few seconds

Allowed methods:

- WebSocket
- Server-Sent Events
- Polling

We evaluate the result, not the method.

Feature 5 — Webhook Simulation (Testing Panel)

Create a page:

`/test-tools`

Purpose:

Simulate a payment gateway confirming subscription.

Buttons required:

1. Reset provider quota to 10 (simulate successful payment)
2. Call webhook multiple times (to test idempotency)
3. Generate 10 leads instantly (to test concurrency)

Important:

- Quota must reset **ONLY** through webhook
 - Calling webhook multiple times must not duplicate effect
 - Webhook logic must not be triggered from normal user UI
-

What We Will Test

We will:

- Submit duplicate leads
 - Generate multiple leads simultaneously
 - Trigger webhook multiple times
 - Leave dashboard open and check real-time updates
 - Check whether quota is respected
 - Verify allocation fairness
-

Submission Requirements

1. GitHub repository
 2. Live demo URL
 3. Setup instructions
 4. Short explanation of:
 - Allocation algorithm
 - How concurrency was handled
 - How webhook idempotency is ensured
-

Evaluation Criteria (Priority Order)

1. Correct provider allocation
 2. Data consistency under concurrency
 3. Webhook safety & idempotency
 4. Real-time dashboard working
 5. Database design quality
 6. Code clarity
-

Important Note

A beautiful UI with incorrect backend logic will be rejected.
A simple UI with correct and safe logic will be accepted.

